

PAMS Merge Guide

Combining the Front-Desk Module into the Main System

Purpose: This guide tells you exactly what to copy and change to get the full working front-desk functionality into the combo system (the git base).

Overview of the Two Systems

	Combo (base — your git repo)	Bzp (working front-desk)
<code>pages/frontdesk_page.py</code>	Static mock data, no real DB calls	Full working dialogs — real DB, 10-min timer, tenant search, leases, complaints
<code>database/db_service.py</code>	Has equipment/worker functions	Has 3 new front-desk functions
<code>database/schema.py</code>	Has <code>equipment</code> table	Missing <code>equipment</code> table
Everything else	✔ Same	✔ Same

Bottom line: 2 files to replace, 1 file to update, 1 table to add.

Step 1 — Replace `pages/frontdesk_page.py`

This is the only major file change. The combo version is a static dashboard with no working buttons. The Bzp version has everything working.

Action: Copy `pages/frontdesk_page.py` from Bzp into the combo repo, replacing the existing file completely.

```
ASD_PAMS-Alpha-B/pages/frontdesk_page.py
→ ASD_PAMS-alpha-tomi-bethel-combo/pages/frontdesk_page.py
```

⚠ One thing to check after copying: the combo `app.py` calls `FrontDeskPage(parent=self)` — this matches the Bzp version, so no changes needed in `app.py`.

Step 2 — Add 3 functions to `database/db_service.py`

The Bzp version has 3 functions the combo doesn't. Add them to the combo's `db_service.py`. Paste each function anywhere after the existing functions — the bottom of the file is fine.

Function 1: `create_apartment_reservation`

Handles the 10-minute apartment lock when assigning a lease.

python

```
def create_apartment_reservation(apt_id: str, user_session_id: str,
                                expires_at) -> str | None:
    """Create a temporary apartment reservation (10 minutes)."""
    rid = _new_id()
    try:
        with get_db() as conn:
            conn.execute(
                """INSERT INTO apartment_reservations
                   (reservation_id, apt_id, user_session_id, expires_at, status)
                   VALUES (?, ?, ?, ?, ?)""",
                (rid, apt_id, user_session_id, expires_at, "active")
            )
            conn.execute(
                "UPDATE apartments SET status = 'reserved_pending' WHERE apt_id = ?"
                (apt_id,)
            )
        return rid
    except Exception:
        return None
```

Function 2: `release_apartment_reservation`

Called when the timer expires or the user cancels.

python

```
def release_apartment_reservation(reservation_id: str) -> bool:
    """Release a temporary apartment reservation."""
    try:
        with get_db() as conn:
            row = conn.execute(
                "SELECT apt_id FROM apartment_reservations WHERE reservation_id = ?"
                (reservation_id,)
            ).fetchone()
            if not row:
                return False
            conn.execute(
                "DELETE FROM apartment_reservations WHERE reservation_id = ?",
                (reservation_id,)
            )
            conn.execute(
                "UPDATE apartments SET status = 'available' WHERE apt_id = ?",
                (row["apt_id"],)
            )
            return True
    except Exception:
        return False
```

Function 3: `get_complaints`

Returns all maintenance tickets that were logged as complaints.

```
python

def get_complaints() -> list[dict]:
    """Get all complaints (tickets with [COMPLAINT] prefix)."""
    with get_db() as conn:
        rows = conn.execute(
            """SELECT * FROM maintenance_tickets
               WHERE description LIKE '%[COMPLAINT]%'
               ORDER BY created_at DESC"""
        ).fetchall()
        return _rows_to_dicts(rows)
```

Step 3 — Add the `apartment_reservations` table to `database/schema.py`

The two new reservation functions write to an `apartment_reservations` table that doesn't exist in either version's schema file yet. Add it to `schema.py` in the combo repo inside the `SCHEMA_SQL` list (or wherever the other `CREATE TABLE` statements are).

```
python
```

```
"""
CREATE TABLE IF NOT EXISTS apartment_reservations (
    reservation_id TEXT PRIMARY KEY,
    apt_id          TEXT NOT NULL REFERENCES apartments(apt_id),
    user_session_id TEXT NOT NULL,
    expires_at      TIMESTAMP NOT NULL,
    status          TEXT NOT NULL DEFAULT 'active'
                    CHECK(status IN ('active', 'expired', 'completed')),
    created_at      TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
"""
```

Step 4 — Keep the `equipment` table in `schema.py`

The combo's `schema.py` has an `equipment` table that the Bzp version is missing. **Do not remove it** — it belongs to Tomisin's maintenance module. It stays as-is.

Step 5 — Delete the old database and re-initialise

Because the schema now has a new table (`apartment_reservations`), the existing `data/pams.db` file needs to be regenerated.

```
bash
```

```
# From the project root:
```

```
del data\pams.db          # Windows
```

```
rm data/pams.db          # Mac/Linux
```

```
python main.py            # This will recreate pams.db with all tables + seed data
```

Step 6 — Git workflow

```
bash
```

```
# On the combo repo (your main branch):
git checkout -b feature/frontdesk-integration

# Copy the files as described above, then:
git add pages/frontdesk_page.py
git add database/db_service.py
git add database/schema.py

git commit -m "feat: integrate working front-desk module (Terrence)

- Replace static frontdesk_page with full working version
- Add create/release_apartment_reservation to db_service
- Add get_complaints to db_service
- Add apartment_reservations table to schema"

git push origin feature/frontdesk-integration
# Then open a pull request into main
```

Quick Verification After Merging

Run the app and log in as front-desk staff. Check these work:

- ☐ **Register Tenant** button opens the registration form
- ☐ **Manage Leases** button shows active leases with expiry warnings
- ☐ **Lease assignment** starts a 10-minute countdown timer
- ☐ **Maintenance Request** button opens the logging dialog
- ☐ **Log Complaint** button works (tags description with `[COMPLAINT]`)
- ☐ **Tenant Inquiry** opens the search dialog

Summary — Files Changed

File	Action
pages/frontdesk_page.py	Replace entirely with Bzp version
database/db_service.py	Add 3 functions from Bzp version
database/schema.py	Add 1 table (apartment_reservations)
data/pams.db	Delete and regenerate
Everything else	No changes needed

